

PLANO DE TESTE PARA PDV

Registro de Mudanças

Versão	Data de Mudança	Por	Descrição
1.0	27/04/2026	Lucas	Criação

1	INTRODUÇÃO	2
1.1	ESCOPO	2
1.1.1	<i>No escopo</i>	2
1.1.2	<i>Fora do escopo</i>	3
1.2	OBJETIVOS DE QUALIDADE.....	3
1.3	PAPÉIS E RESPONSABILIDADES	3
2	METODOLOGIA DE TESTE	3
2.1	FASES DE TESTE	4
2.2	CRITÉRIOS DE SUSPENSÃO E REQUISITOS DE RETOMADA.....	4
2.3	COMPLETUDE DO TESTE	4
2.4	ATIVIDADES DO PROJETO, ESTIMATIVAS E CRONOGRAMA	4
3	ENTREGÁVEIS DE TESTE.....	5
4	NECESSIDADES DE RECURSOS E AMBIENTE	5
4.1	FERRAMENTAS DE TESTE.....	5
4.2	AMBIENTE DE TESTE	5
5	TERMOS / ACRÔNIMOS.....	6

1 Introdução

Este plano de teste define as estratégias, o processo, o fluxo de trabalho e as metodologias utilizados para garantir a qualidade do sistema PDV (Ponto de Venda). A abordagem adotada é híbrida, validando tanto as regras de negócio na camada de serviços (via testes automatizados com JUnit/Mockito) quanto a experiência e funcionalidade da interface do usuário (via testes manuais baseados em requisitos)

1.1 Escopo

1.1.1 No escopo

O escopo define os recursos e requisitos funcionais do software que serão testados nesta etapa do projeto. A validação abrangerá os métodos de serviço back-end responsáveis pelo controle de Pessoas, Vendas e Produtos no sistema PDV.

Nome do Módulo	Papéis aplicáveis	Descrição
Serviços Automatizados (Back-end)	PessoaService	Validações de cadastro, regras de filtro e integração de listagem.
Serviços Automatizados (Back-end)	VendaService	Contagem de vendas, bloqueios de fecho (valores zerados/negativos), listagem e remoção de produtos.
Serviços Automatizados (Back-end)	ProdutoService	Inserção de produtos e validação da listagem.
Testes Funcionais Manuais	RF01	Registrar Venda
Testes Funcionais Manuais	RF02	Listar e Filtrar Vendas (abertas e fechadas)
Testes Funcionais Manuais	RF03	Registrar Produto (com e sem isenção/substituição tributária)
Testes Funcionais Manuais	RF04	Ajustar Estoque Manualmente (adicionar e remover itens)
Testes Funcionais Manuais	RF05	Controle de Abertura e Fecho de Caixa
Testes Funcionais Manuais	RF06	Gerir Movimentações de Caixa (Suprimento, Transferência e Retirada)
Testes Funcionais Manuais	RF07	Autenticação de Utilizadores (válidos e inexistentes)
Testes Funcionais Manuais	RF08	Gestão de Sessão (Logout)

1.1.2 Fora do escopo

Os seguintes recursos e requisitos funcionais **NÃO** serão testados nesta rodada:

- Testes de Carga, Stresse e Performance do servidor local ou base de dados.
- Validação de compatibilidade em múltiplos navegadores (Cross-browser) ou dispositivos móveis.
- Testes de segurança contra invasões e testes de penetração (PenTest).

1.2 Objetivos de Qualidade

Os objetivos que planeamos alcançar com estes testes são:

- Certificar-se de que o software sob teste (SUT) está em conformidade com as regras de negócio de vendas, controlo financeiro (caixa) e controlo de stock.
- Garantir a estabilidade da camada de back-end por meio da execução de testes unitários automatizados.
- Garantir que os defeitos de fluxo crítico identificados (como falhas no ecrã de pagamento ou falhas na atualização de stock) sejam corrigidos antes de o sistema entrar em operação.

1.3 Papéis e Responsabilidades

O projeto deve usar membros terceirizados como testadores para economizar o custo do projeto.

No.	Membro	Tarefas
1	Gestor de Teste / PO	Responsável por priorizar as correções de bugs, definir o escopo das Sprints e aprovar a completude dos testes.
2	Projetista de Teste	Responsáveis pela criação e manutenção dos scripts de teste automatizado (JUnit/Mockito) na camada de Service, e pela correção dos defeitos reportados
3	Analista de QA (Testador)	Responsável por planear e executar os cenários de testes manuais no ambiente http://localhost:8080 , bem como registar os resultados no TestLink (ex: testador admin3)

2 Metodologia de Teste

A metodologia de teste adotada é baseada em práticas de desenvolvimento **Ágil**. O processo combina testes unitários e de integração contínuos no código-fonte em conjunto com testes funcionais exploratórios e baseados em casos de teste na interface final.

2.1 Fases de Teste

As seguintes fases serão executadas no software sob teste (SUT):

1. **Testes de Unidade e Integração:** Executados diretamente no código Java (ServiceTest).
2. **Testes Funcionais/Sistema:** Executados manualmente na interface web iniciando sessão como gerente/123.
3. **Testes de Regressão:** Executados após a correção dos erros apontados na fase de triagem.

2.2 Triagem de Erros

A resolução dos erros identificados seguirá uma priorização baseada no seu impacto no sistema:

- **Alta Prioridade (Bloqueadores):** Devem ser corrigidos imediatamente. Exemplo identificado no RF01 (Caso 00111-9): O ecrã de pagamento apresenta um título sem opções, impossibilitando a conclusão da venda.
- **Média Prioridade (Funcionais Graves):** Devem ser corrigidos na Sprint atual. Exemplo identificado no RF04 (Casos 00111-15 e 16): Ao ajustar o stock manualmente (inserir ou remover), o sistema processa a ação, mas a quantidade do item no stock permanece zero e não é atualizada.

2.3 Critérios de Suspensão e Requisitos de Retomada

☐ **Critérios de Suspensão:** Os testes em ecrã serão suspensos caso o ambiente principal (<http://localhost:8080>) fique indisponível, ou se erros críticos (como falha no RF07 - Autenticação) impedirem o acesso ao sistema web.

☐ **Requisitos de Retomada:** Os testes serão retomados assim que a equipa de desenvolvimento restabelecer o ambiente local ou realizar o deploy da correção do erro bloqueador.

2.4 Completude do Teste

O ciclo de testes será considerado completo quando:

- 100% dos testes unitários de backend (PessoaService, VendaService, ProdutoService) passarem sem exceções.
- Todos os 15 casos de teste manuais mapeados (00111-9 ao 00111-23) forem executados na ferramenta TestLink.
- Todos os erros abertos com status "Falhado" (RF01 e RF04) forem corrigidos e validados por um novo teste.

2.5 Atividades do projeto, estimativas e cronograma

Tarefa	Membros	Estimativa de esforço
Planejamento de Testes	Projetista de teste	2 homens/hora
Executar os testes Automatizados	Testador, Administrador de teste	Contínuo
Execução de Testes Manuais	Testador	10 homens/hora
Entregar os testes		20 homens/hora
Total		280 homens/hora

3 Entregáveis de Teste

Os artefactos resultantes desta fase de testes incluem:

- Plano de Teste (Este documento).
- Scripts de automação (PessoaServiceTest.java, VendaServiceTest.java, ProdutoServiceTest.java).
- Casos de teste documentados e registados no TestLink.
- Relatório de Erros e status de execução (Ex: Passou / Falhado)

4 Necessidades de Recursos e Ambiente

4.1 Ferramentas de Teste

No.	Recursos	Descrição
1	Servidor	Necessário um servidor de banco de dados com MySQL instalado e um servidor Web com Apache instalado
2	Ferramenta de teste	TestLink (Hospedado no ambiente da UFF)
3	Automação de Testes (Back-end)	JUnit 4, Mockito, Spring Boot Test

4.2 Ambiente de Teste

Ambiente de teste a ser configurado de acordo com a figura abaixo

- **Acesso do Sistema:** Aplicação a correr no host `http://localhost:8080`.
- **Massa de Dados Inicial:** Exige produtos registados, itens em stock e utilizadores configurados (ex: credencial gerente/123).
- **Softwares de Base:** Java (JDK), IDE (Eclipse/IntelliJ), navegador web padrão para interface e Windows 8 ou superior.

Setup

- Clone o projeto em uma pasta vazia
- Instale Maven no seu computador
- Instale Java JDK8 (Pode ser a versão Temurin JDK 8) e adicione ao seu JAVA_HOME
- Com o VScode (ou algum terminal similar aberto), rode os seguintes comandos em sequência em um terminal:
 - `docker-compose down`
 - `mvn clean`
 - `export MAVEN_OPTS="-Xmx512m -XX:MaxPermSize=256m"`
 - `mvn clean package -DskipTests`
 - `docker-compose up --build` (Aguarde até o app pdv iniciar)
- Abra um terminal adicional e rode:
 - `mvn test -DforkCount=0`

5 Termos / Acrônimos

TERMO / ACRÔNIMO	DEFINIÇÃO
API	Application Program Interface
SUT	Software Under Test (Software Sob Teste)
QA	Quality Assurance (Garantia de Qualidade)
PDV	Ponto de Venda
RF	Requisito Funcional